



DCBLSTM—Deep Convolutional Bidirectional Long Short-Term Memory neural network for Q8 secondary protein structure prediction

Suvidhi Banthia ^{a,1}, Adam McKenna ^{b,1}, Shailendra Kumar Tiwari ^c, Sandhya P.N. Dubey ^{a,1,*}

^a Department of Data Science and Computer Applications, Manipal Institute of Technology, Manipal Academy of Higher Education (MAHE), Manipal, 576104, Karnataka, India

^b School of Electronics, Electrical Engineering and Computer Science, Queen's University of Belfast, Belfast, BT7 1NN, United Kingdom

^c Department of Electronics and Communication Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education (MAHE), Manipal, 576104, Karnataka, India

ARTICLE INFO

Dataset link: <https://github.com/SuvidhiBanthia/a/DCBLSTM>

Keywords:

Convolutional neural network
Deep learning
Long short-term memory
Machine learning
Protein folding
Protein secondary structure

ABSTRACT

Protein secondary structure prediction involves determining a protein's secondary structure from its primary amino acid sequence, serving as a critical step toward tertiary structure prediction. This, in turn, is essential for applications in drug design, protein engineering, and genetic research. Given the complexity of this task, advanced methods such as Gated Recurrent Units (GRUs) and Long Short-Term Memory (LSTM) networks are often employed, as they effectively capture long-range dependencies between amino acids, thereby improving prediction accuracy.

In this study, we utilized the latter, specifically Bidirectional Long Short-Term Memory (BLSTM) networks, which process protein sequences in both forward and backward directions. This bidirectional processing has shown considerable promise in this domain. To further enhance local feature extraction, the network architecture incorporates a local feature encoding and extraction module consisting of three 1-dimensional convolutional layers, designed to capture dependencies between adjacent amino acids.

Several optimization and regularization techniques were applied to refine the model, including batch normalization, kernel initialization, kernel regularization, dropout, and pooling layers. Optimal values for each parameter were identified through meticulous hyperparameter tuning. The final proposed model, termed Deep Convolutional BLSTM (DCBLSTM), was evaluated on three publicly available and widely recognized datasets: CB513, CASP10, and CASP11. For Q8-state classification, the model achieved accuracies of 88.9%, 83.9%, and 84.3%, respectively, on these datasets. These results demonstrate that the proposed model delivers state-of-the-art accuracy, outperforming several existing benchmark models. The consistently high accuracy highlights the effectiveness and robustness of the DCBLSTM model for protein secondary structure prediction.

1. Introduction

Protein Secondary Structure Prediction (PSSP) involves predicting a protein's secondary structure from its primary sequence, thereby aiding in tertiary structure determination. PSSP is a crucial objective in bioinformatics, with a significant and ongoing relevance in both medical and biotechnological fields [1]. It plays a pivotal role in various applications, including understanding protein folding mechanisms and their dynamics [2,3], sequence alignment, function prediction [4,5], and identifying genetic mutations associated with diseases [6–8]. One notable example is sickle cell anemia, which results from the

substitution of a single amino acid in the hemoglobin molecule [9]. This condition has also been associated with various cancers, further highlighting the importance of continued research in this field [10].

The protein secondary structures are classified into three main categories: H (helices), E (strands), and C (coils). The Define Secondary Structure of Proteins (DSSP) program extends this classification into eight categories to capture more detailed structural features [11]. Traditional three-category (Q3) PSSP methods, though often accurate, provide limited structural insight compared to the more detailed eight-category (Q8) approach. However, Q8 methodologies present

* Corresponding author.

E-mail addresses: suvidhi.banthia@learner.manipal.edu (S. Banthia), amckenna41@qub.ac.uk (A. McKenna), sk.tiwari@manipal.edu (S.K. Tiwari), sandhya.dubey@manipal.edu (S.P.N. Dubey).

¹ These authors contributed equally to this work.

<https://doi.org/10.1016/j.combiomed.2025.110457>

Received 5 September 2024; Received in revised form 23 May 2025; Accepted 24 May 2025

Available online 23 June 2025

0010-4825/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

challenges, including reduced precision and increased computational demands [12,13], which has driven the adoption of deep learning techniques to enhance PSSP performance. This research primarily focuses on deep learning-based methods for Q8 PSSP, leveraging recent advancements in computational biology.

Recognizing the importance of local context — referring to interactions between adjacent residues — Convolutional Neural Networks (CNNs) have proven effective in capturing local dependencies within protein sequences [13–15]. Our model utilizes a CNN architecture comprising three one-dimensional convolutional layers, optimized through dropout, max pooling, and batch normalization. The initial layer encodes low-level features, while subsequent layers progressively capture more complex and abstract feature representations.

Additionally, accounting for long-distance dependencies within protein sequences is essential, as interactions between distant amino acids can significantly influence the folding process [16]. Similar long-range dependencies are observed in fields such as speech recognition and language translation [17,18]. To address this, recurrent neural networks — including Long Short-Term Memory (LSTM) units and Gated Recurrent Units (GRUs) — have been employed to incorporate a temporal dimension into the prediction process, effectively mapping these dependencies [12,19,20].

Bidirectional LSTMs effectively capture both past and future contextual information within a sequence. While GRUs provide a less computationally intensive alternative [21], LSTMs have consistently demonstrated superior performance in PSSP tasks [12,19–21]. Consequently, we incorporated LSTMs into our final model.

By combining CNNs for local feature encoding with LSTMs for capturing long-distance dependencies, we propose the Deep Convolutional Bidirectional LSTM (DCBLSTM) to improve the precision and reliability of Q8 PSSP predictions. The model was trained on the publicly available CullPDB6133 dataset and evaluated using the CB513, CASP10, and CASP11 datasets. Further details about these datasets are provided in the section Training, Testing, and Validation Data. Our model achieves state-of-the-art accuracy, outperforming existing PSSP benchmark methods.

1.1. Background

Proteins are composed of one or more polypeptide chains of amino acid residues and perform a wide range of essential functions in living organisms. These include catalyzing enzymatic reactions, transmitting intracellular signals, serving as antibodies, and providing critical structural support [22].

A protein is organized into four primary structural levels: primary, secondary, tertiary, and quaternary. The primary structure refers to the linear sequence of amino acids, while the secondary structure includes local configurations such as alpha-helices and beta-strands or beta-sheets. The tertiary structure represents the overall three-dimensional arrangement of the protein, incorporating one or more secondary structure elements [23]. The quaternary structure involves the assembly of multiple polypeptide chains into a functional complex [24]. Additionally, quinary structures — though less commonly discussed — describe the dynamic characteristics of protein surfaces shaped by evolutionary adaptation, influencing surface features in response to the cell's physiological environment [25].

This work contributes to a long-standing research effort in protein structure sequencing and prediction, a field that dates back to the sequencing of insulin by Sanger and Tuppy in 1949 [26] and the identification of alpha helices and beta sheets by Pauling and Corey in 1951 [27,28]. These discoveries significantly advanced the understanding of protein secondary structures and also influenced Watson and Crick's 1953 double-helix model of DNA [29]. Their foundational work paved the way for numerous breakthroughs in molecular biology, including DNA replication, genetic engineering, and DNA profiling [30–33].

1.2. Limitations

Protein Secondary Structure Prediction (PSP) methods have significantly advanced over the past 30 years, but their accuracy has begun to plateau, approaching a theoretical limit imposed by current machine learning approaches. This limitation may arise from difficulties in accurately assigning DSSP labels at the boundaries of secondary structures, which are affected by local conformational variations and crystal packing effects [34]. Furthermore, the arbitrary division into three structural states and approximately 12% variability in secondary structure assignments due to structural homologies restrict the maximum achievable accuracy to around 90% [35,36].

Another limitation of PSP lies in its cost: sequencing a species' genome is significantly cheaper (under \$10,000) compared to determining the structure of a single protein, which can cost around \$100,000 [37]. Consequently, there is a vast difference in the volume of available data, with over 200 million genomic sequences stored in GenBank [38] compared to approximately 100,000 protein structures in the Protein Data Bank [39]. This disparity underscores the challenges of protein analysis, as experimental methods like X-ray crystallography face limitations including the need for crystallizable samples, constraints on protein size, and the inability to observe proteins in their native environments—factors that can affect the accuracy of the resulting structures [40].

Furthermore, chameleon sequences pose a challenge to PSP due to their variability in secondary structure prediction. These peptides, experimentally observed to adopt different secondary structures in distinct proteins, can be up to ten residues long [41]. Research suggests that applying a sliding window larger than 20 residues can effectively identify such sequences. To aid in this effort, the ChSeq database catalogs over 19,000 chameleon sequences [42].

A final, and arguably the most critical, constraint is the ability to map and capture non-local interactions within protein sequences, which forms the foundation of this paper. As noted earlier, non-local interactions involve contacts between residues that are distant in the primary sequence but may be spatially close in the three-dimensional structure. While local sequence information provides some insight, it accounts for only about a quarter of the data necessary to accurately predict a protein's secondary structure [43]. Studies have shown that prediction accuracy for helical and sheet-like structures declines as the distance between interacting amino acids increases [44].

2. Related works

Protein Secondary Structure Prediction (PSP) has been an active research area for over 65 years, with approximately 266 methods reported between 1973 and 2016 [45]. Early efforts primarily focused on identifying alpha helices using helix-coil transition models [46], achieving accuracies of around 60–65

The Chou–Fasman method is among the earliest algorithms developed for predicting protein secondary structure [47]. It analyzes amino acid frequencies to estimate their propensity to form alpha helices, beta strands, or turns. Achieving around 50%–60% accuracy, this first-generation approach relied on statistical tendencies of amino acids toward specific secondary structures [45].

The GOR method, an extension of the Chou–Fasman approach, employed probabilistic inference to predict protein secondary structures by analyzing the likelihood of amino acid arrangements along with their neighboring residues. This second-generation methodology improved upon Chou–Fasman by incorporating adjacent residue context, achieving an accuracy of approximately 65% [48].

The transition from first- to second-generation PSP methods led to notable improvements in accuracy. However, the advent of third-generation techniques — characterized by the use of sequence profiles derived from multiple sequence alignments — has become the standard

approach in contemporary PSP. These methods have achieved accuracies exceeding 70% and are approaching the theoretical maximum of approximately 90% [49,50].

Much of the recent progress in PSP methods has been driven by the biannual Critical Assessment of protein Structure Prediction (CASP) meetings [51]. These gatherings have documented a steady shift from traditional PSP techniques toward machine learning approaches, especially deep learning. Various machine learning algorithms have contributed to improved results, including Hidden Markov Models [45], support vector machines [52], random forests [46], linear regression [53], and k-nearest neighbors (KNN) [47]. More recent PSP methods have increasingly focused on neural network architectures.

SSPro8 utilized an ensemble of bidirectional recurrent neural networks trained on a relatively small dataset of 1180 proteins, achieving Q3 and Q8 accuracies of approximately 73% and 63%, respectively [48]. In 2014, the SCORPION algorithm employed a three-stage feed-forward network incorporating multiple sequence alignments (PSSM), reaching Q3 and Q8 accuracies of around 83% and 71%, respectively [54]. SPIDER2, based on a deep artificial neural network, achieved roughly 82% Q3 accuracy [49]. The ICML2014 model, built upon a deep stochastic neural network, attained about 72% Q8 accuracy [50]. MUFOLD-SS used a deep neural network architecture called inception-inside-inception, achieving 86% Q3 and 76% Q8 accuracy [51]. The DeepCNF algorithm featured a deep neural network with five hidden layers, each containing 100 neurons and a window size of 11, obtaining 85% Q3 and 75% Q8 accuracies [13]. CNNH_PSS leveraged a multi-scale convolutional neural network combined with a fully connected Softmax layer, achieving 70% Q8 accuracy [55]. Many of these methods are classified as third-generation PSP techniques, characterized by their use of evolutionary information derived from the alignment of multiple homologous sequences.

In this paper, the approach combines convolutional layers with a bidirectional recurrent network and fully connected layers. Similar approaches have shown encouraging results. Zhang et al. used 1D convolutional layers with bidirectional gated recurrent units (GRUs), with an accuracy of 71.4% for Q8 prediction obtained on the CB513 benchmark dataset [19]. SPIDER3 possessed bidirectional LSTM units within a neural network, with the entire protein sequence being input to the network; it achieved a Q3 score of 84% [21]. The DeepACLSTM algorithm had a Q8 accuracy of 75%. This algorithm was built using a local feature encoding module and a long-distance dependency encoding module, realized through bidirectional LSTMs [12]. CRRNN involved a convolutional, residual, and recurrent neural network, achieving a Q3 and Q8 score of 86% and 73%, respectively. In the same paper, an ensemble of trained CRRNN models was integrated for an algorithm called eCRRNN, where accuracy improved to 88% and 76% [19]. The DCRNN algorithm used multi-scale convolution layers linked by three stacked bidirectional recurrent network layers, achieving a 70% Q8 accuracy on the CB513 benchmark [56]. A CNN-SVM ensemble achieved 97.91% accuracy with 20% of residues, 85.13% with 80%, and 78.02% with 100% on the CullPDB6133 dataset [57]. The transformer encoder-based model TE_SS and Ankh protein language model achieved a Q8 accuracy of 79.08% and 77.57% on the TEST2016 and TEST2018 datasets, respectively [58].

3. Methodology *Compares other methods*

3.1. Model architecture

The architecture consists of three main components: a one-dimensional Convolutional Neural Network (CNN), a two-layered bidirectional Long Short-Term Memory (LSTM) Recurrent Neural Network (RNN), and a dense fully connected Deep Neural Network (DNN). Each of these components has demonstrated effectiveness individually in related domains—for example, CNNs in Large Vocabulary Continuous Speech Recognition (LVCSR), DNNs in speech recognition, and RNNs in

Table 1

Execution times on PARAM Utkarsh supercomputer.

Execution Time	Value
Training the model	2588.44 s (~ 43 min)
Testing on CB513	8.03 s
Testing on CASP 10	3.085 s
Testing on CASP 11	1.76 s

computer vision [59–61]. Moreover, combining these three components into a unified network has proven advantageous in several studies [12–15].

The model takes as input the protein sequence, which is vectorized into embeddings and a protein profile, both having dimensions of (700, 21). These features are concatenated to form the input for the subsequent stage. It has been observed that CNNs alone are not optimal for natural language processing (NLP) tasks such as protein secondary structure prediction (PSP). This limitation arises from CNNs' inability to effectively model long-distance dependencies and capture contextual information within sequences. To overcome this drawback, approaches such as *coupling time-delayed neural networks (TDNN)* and *dynamic convolutional neural networks (DCNN)* have been proposed [62,63].

As discussed in the introductory section, long-distance co-dependencies between amino acids frequently occur within protein sequences. Consequently, given the previously mentioned limitations of CNNs in capturing such dependencies, they should not be employed in isolation for this type of task. The concatenated input is subsequently passed through three stacked one-dimensional convolutional layers with progressively increasing filter sizes of 16, 32, and 64, and kernel sizes of 3, 5, and 7, respectively. Following each convolutional layer, a feature map is generated and processed through several optimization and regularization layers, including batch normalization, dropout (rate 0.3), and max pooling. These steps help stabilize the training process and mitigate overfitting. The multi-scale convolutional structure enables the extraction of local features at varying levels of granularity. Finally, the outputs from these convolutional layers are concatenated to produce a unified convolutional feature map.

The output from the CNN was fed into a sequence of two stacked bidirectional LSTM (BLSTM) layers, each comprising 200 units. These layers enable the model to capture long-distance dependencies and account for temporal variations within the protein sequence. By processing information in both forward and backward directions, the BLSTM layers enhance the model's ability to comprehend contextual relationships across the entire sequence.

Following the BLSTM layers, the output is passed through two fully connected (dense) layers containing 300 and 150 units, respectively. Each dense layer is followed by a dropout layer with a rate of 0.3 to mitigate overfitting. The final layer is a dense output layer with 8 units, corresponding to the classification of protein secondary structure categories. The overall model architecture is illustrated in Fig. 1. In total, the model comprises 3,190,337 parameters, of which 3,190,113 are trainable and 224 are non-trainable.

3.1.1. Implementing network

The model was implemented using TensorFlow and Keras. The LSTM recurrent layers proved to be the most computationally intensive components, primarily due to their bidirectional architecture and the requirement for backward propagation through time. This computational complexity, combined with the size of the dataset, made local training and evaluation challenging, necessitating access to a more robust computing infrastructure.

To facilitate this, the model architecture was configured for training and evaluation on the PARAM Utkarsh supercomputer—a High-Performance Computing (HPC) system specifically developed for Micro, Small, and Medium Enterprises (MSMEs) under India's National Supercomputing Mission (NSM). Located at C-DAC Bangalore, PARAM

Table 2
Details about hardware used on PARAM Utkarsh supercomputer.

Header for property	Property	Value
Details about 'CPU only nodes'	Processor	2* Intel Xeon Platinum 8268
	Cores	48, 2.9 GHz
	Memory	192 GB, DDR4 2933 MHz
	SSD	480 GB (local scratch) per node
	Total Cores	3600
	Total Memory	14400 GB
Specific to our job (run on a CPU only node)	Slurm Cluster name	paramutkarsh
	Slurm Job partition	cpu
	Slurm CPUs on node	2
	No. of Tasks per node	2

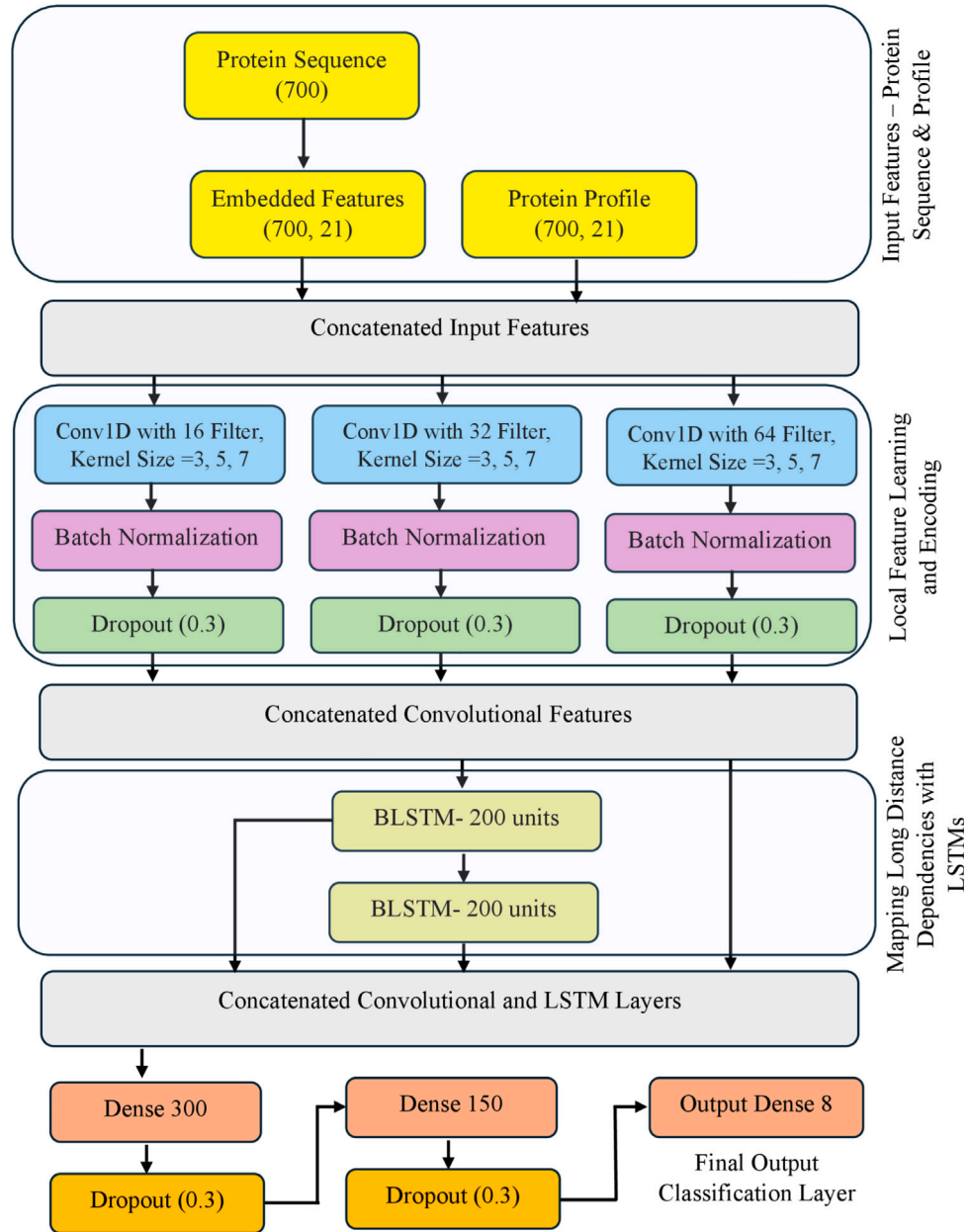


Fig. 1. Model Architecture of DCBLSTM showcasing the input features, CNN layers, BLSTM and fully connected layers.

Utkarsh delivers exceptional computational power with over 50,000 cores and a peak performance of 838 Teraflops. The system integrates Intel Cascade Lake processors and NVIDIA Tesla V100 GPUs, connected via a high-speed 100 Gbps Infiniband network. This infrastructure

enabled the efficient training of complex models such as ours, significantly reducing training time compared to local machines [64]. Details of execution times and hardware configurations are presented in Table 1 and Table 2, respectively.

3.2. Feature engineering

3.2.1. Input features

The input features selected for the network were the protein sequence and the protein profile. As demonstrated in the DeepACLSTM study, which adopts a similar approach to this work, combining both of these input features yields the best predictive performance [12]. In that study, using only protein profile features resulted in approximately a 10% increase in accuracy compared to using only protein sequence features. Furthermore, incorporating both features together provided an additional marginal improvement of about 1%. The protein sequence features are represented as a 21-dimensional vector indicating the type of amino acid at each position in the protein. Meanwhile, the protein profile is also a 21-dimensional vector, derived from a position-specific scoring matrix (PSSM), which captures evolutionary information by reflecting the likelihood of each amino acid occurring at a specific position.

The protein sequence features underwent an embedding process to convert their initial sparse representation into a denser, continuous 21-dimensional vector. This transformation allowed the model to capture richer semantic relationships between amino acids. After embedding, these protein sequence features were concatenated with the protein profile features—also 21-dimensional vectors derived from the position-specific scoring matrix (PSSM). The combined feature set was then passed into the network's hidden layers for further processing.

3.2.2. Feature encoding and learning through convolutions

Three stacked one-dimensional convolutional layers were incorporated into the architecture, as illustrated in Fig. 1. One-dimensional convolutions are particularly effective for sequence-based tasks, including sentiment analysis, time series forecasting, and sequence prediction [65]. In the context of PSP, CNNs have demonstrated notable success, which motivated their inclusion in our model architecture [12, 13, 55, 66]. A stacked configuration was employed for the convolutional layers, as it has been shown that layering multiple convolutional layers not only reduces the overall parameter space but also enables the model to learn more complex, hierarchical, and nonlinear feature representations.

While convolutional layers are highly effective at capturing local dependencies between adjacent amino acids, they are limited in their ability to model interactions between distant residues. One-dimensional CNNs, however, offer a significant computational advantage over their two- and three-dimensional counterparts by relying on simpler array operations, which reduces overall model complexity and training time [67].

The input to the CNN is a concatenation of the feature vectors. Following the three convolutional layers, the output is normalized using batch normalization, after which it is passed through a Rectified Linear Unit (ReLU) activation layer, followed by a dropout layer, and finally a max pooling layer. The ordering of layers before and after batch normalization has been the subject of much discussion. After various assessments of performance with different layer orderings, this specific sequence of layers was found to yield optimal results. Finally, the output from the max pooling layers is concatenated and fed into the RNN component of the network.

Given the 1-dimensional sequence data:

$$X = (x_1, x_2, x_3, \dots, x_{t-1}, x_t, x_{t+1}, \dots, x_{n-1}, x_n) \quad (1)$$

where X denotes a protein sequence with n amino-acid residues and $x_i = (x_{i1}, x_{i2}, \dots, x_{ij}, \dots, x_{im})$ represents the feature vector of the i th residue, local contextual dependencies exist of a residue x_i , meaning it is contingent on its adjacent amino acids in the forward and backward direction of sequence X .

The residues are convolved by the convolution operation:

$$h_i = f((W * x_{i:i+k-1}) + b) \quad (2)$$

where $*$ denotes the convolutional operation, k represents the kernel size, b is the bias term, and f is a nonlinear function such as sigmoid, tanh, or ReLU. In our model, we chose ReLU as our activation function, and the kernel size was set to 7 for each convolutional layer. The output of this convolution operation is a feature map c :

$$C^1 = [C_1^1, C_2^1, C_3^1, \dots, C_{N-2}^1, C_{N-1}^1, C_N^1] \quad (3)$$

Three convolutional operations are completed for the three layers generating three feature maps in total. Each of the generated feature maps are then passed through a batch normalization layer:

$$\hat{x}_i = \frac{x_i - \mu_\beta}{\sqrt{\sigma_\beta^2 + \epsilon}} \quad (4)$$

where x_i is the input value generated from the convolutional feature map, μ_β is the mean of the mini-batch, σ_β^2 is the mini-batch variance, and i is the normalized output. The normalized output is then scaled and shifted:

$$\hat{y}_i = \gamma \hat{x}_i + \beta \quad (5)$$

where γ is a learned scaling factor and β is a learned offset factor.

Dropout is then applied to the normalized output:

$$\hat{w}_j = \begin{cases} \hat{y}_{ij}, & \text{with } P(C) \\ 0, & \text{otherwise} \end{cases} \quad (6)$$

where $P(C)$ is the probability C to keep a row in the normalized weight matrix, $\hat{y}_{ij}$ the real row in the weight matrix before dropout and $\hat{w}_j$ is the diluted row of the weight matrix, where dilution is the process of thinning the layer weights [68].

The convolutional layers contain different configurable metrics and parameters. Some of the more important convolution parameters are discussed below.

Filters: The optimal numbers of filters for the layers in the model were 16, 32, and 64, respectively. Increasing the number of filters across convolutional layers is a common practice, as the complexity of feature extraction increases deeper in the network. The first layer encodes basic, low-level features, while the third layer captures more complex feature abstractions [69].

Padding: Padding was applied within the convolutional layers to preserve the dimensionality and size of the output, keeping it consistent with the input. Implementing padding was also necessary for the concatenation of the convolutional layers, each of which uses a different filter size; without padding, the output dimensions would not align [70].

Stride: A stride of 1 is used as the default setting for the convolutional layers. Using a small stride increases computational cost, as the convolution operation is performed on more segments of the input sequence. However, this approach allows for the extraction and encoding of more detailed feature information [71].

Kernel Size: The selection of the kernel size was based on similar literature for PSP problems that also contained convolutional layers and simple trial and error via hyperparameter tuning [12, 13, 15]. Choosing a kernel size in a convolutional neural network involves a trade-off: smaller kernels are faster with lower memory usage, while larger kernels capture more complex features. Larger filter sizes help better extract and encode the local context between amino acids in protein sequences [56]. The model showed the statement was valid up to a size of seven, with no increase in accuracy beyond that and a decrease past eleven.

All other specific convolutional layer parameters were kept at their default values throughout training. Further details about these additional parameters can be found in the Keras documentation [72].

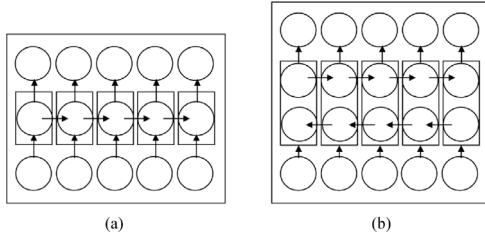


Fig. 2. Structural overview indicating the fundamental difference between (a) unidirectional RNN and (b) bidirectional RNN [75].

3.3. Pooling layers

Pooling is a process similar to convolution, where an operation and filter size are selected and applied to the feature map. Pooling helps the model achieve translation invariance for small shifts in the input, i.e., the output stays similar despite minor input movements [73]. The pooling layer chosen for our model was Max pooling. It calculates the maximum value within the pooling window of the feature map. Max pooling proved to be the most successful and effective pooling layer to use with our model. To maximize model performance, we employed max pooling after batch normalization, activation, and dropout, following the convolutional layers. Other pooling layers attempted with the model were Average pooling, Global max pooling, and Global average pooling.

Max Pooling is represented as:

$$h'_{xy} = \max_{i=0,\dots,s; j=0,\dots,s} h^{l-1}(x+i)(y+j) \quad (7)$$

where h^l is the activation map of layer l . x and y are the spatial coordinates within the activation map and s represents the size of the pooling window.

Following the maximum pooling of each convolutional layer, the output is concatenated into one feature matrix:

$$c = \text{concat}(h'_{xy}{}^1, h'_{xy}{}^2 + h'_{xy}{}^3) \quad (8)$$

Prior to the recurrent component of the network, the concatenated convolutional features are passed through the first fully-connected dense layer of the network:

$$fc^1 = f(W * c + b) \quad (9)$$

where fc^1 represents the output of the first fully-connected dense layer in the network, f is a linear/non-linear activation function, c is the concatenated convolved features, and b is the bias vector.

3.4. Recurrent long-distance encoding with LSTMs

3.4.1. Recurrent neural networks

RNNs possess memory that captures contextual information about the outputs from the previous elements of the sequence, making them helpful in use cases where elements of a sequence may have long-distance dependencies on other elements [74]. The most popular types of RNNs are the Gated Recurrent Unit (GRU) and Long Short-Term Memory units (LSTM), and it is the LSTMs that are the focus of our model.

Typically, RNN applications utilize a unidirectional architecture, where the input sequence is processed solely in the forward direction. For bidirectional RNNs, the main difference is that the reverse of the input sequence is fed into the RNN in the backward temporal direction. The benefit of this approach is the ability to consider the future context of the input sequence, whereas unidirectional RNNs can only consider the context of previous inputs [76]. Fig. 2 shows the fundamental difference between unidirectional and bidirectional RNNs.

RNNs have allowed for significant progress to be made in PSP, this can be attributed to the inevitable long-distance dependencies that exist between residues within long protein sequences [12,19,21,48,56,66].

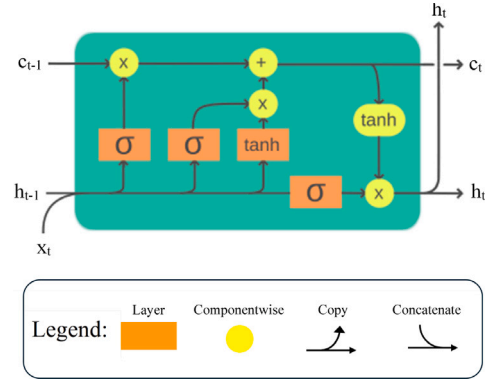


Fig. 3. Visualization and Functioning of the LSTM Cell showcasing the sigmoid functions representing the three gates [78].

3.4.2. Bidirectional LSTMs

Long Short-Term Memory (LSTM) units were initially proposed in 1997 by Hochreiter et al. with the aim of overcoming the problem of vanishing gradients in RNNs [76].

LSTMs have three gates: the forget gate, input gate, and output gate. The forget gate determines what information to discard, using a sigmoid function to produce values between 0 (meaning forget) and 1 (meaning remember). The input gate updates the cell state, also using a sigmoid function to decide whether to update (1) or not (0). Lastly, the output gate controls how much of the cell's value is used for the next hidden state and output activation [77]. Fig. 3 visualizes the LSTM cell and illustrates its functioning.

LSTMs have shown continued prominence in recent PSP-related papers [12,21,66,79]. One of the main benefits of LSTMs is that they are ideal for mapping long-distance dependencies for sequence-related problems, which we take advantage of in our model [17,80,81].

A specific variation of an LSTM was spawned after bidirectional recurrent neural networks (BRNN) became prominent after their introduction by Schuster et al. [82]. It has been demonstrated that bidirectional LSTMs are more effective than unidirectional LSTMs in many fields, including some of the previously mentioned examples of speech recognition, time-series forecasting, and phoneme classification [17,80,83].

An LSTM unit featuring a forget gate can be formally defined using the below formulas:

$$f_t = \sigma(W_f \cdot x_t + U_f \cdot h_{t-1} + b_f) \quad (10)$$

$$i_t = \sigma(W_i \cdot x_t + U_i \cdot h_{t-1} + b_i) \quad (11)$$

$$o_t = \sigma(W_o \cdot x_t + U_o \cdot h_{t-1} + b_o) \quad (12)$$

$$\tilde{c}_t = \tanh(W_c \cdot x_t + U_c \cdot h_{t-1} + b_c) \quad (13)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (14)$$

$$h_t = o_t \odot \tanh(c_t) \quad (15)$$

where W_x and U_x are matrices containing the input weights and recurrent connections. They can either be the weight matrices of the input gate (i), output gate (o), and the forget gate (f) of the LSTM cell. f_t , i_t , o_t and $\tilde{c}_t$ are the activation vectors for the forget, input, output gates, and cell input, respectively. c_t represents the cell state vector, and h_t is the hidden state vector or the output vector of the LSTM unit. α , b , and $\odot$ represent the sigmoid function, bias, and elementwise multiplication. The input sequence in the forward direction in the bidirectional LSTMs (BLSTM), $\bar{h}$, is iteratively calculated from time $T - n$ to time $T - 1$. The

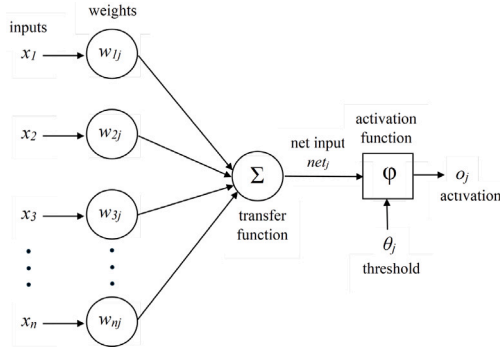


Fig. 4. Role of an activation Function and its position within a neural network [85].

backward direction, reversed sequence input, $\bar{h}$, is calculated from time $T - n$ to $T - 1$ [84]. An output vector y_T is generated from the BLSTM layers where each vector element is calculated by the following:

$$y_t = f(\bar{h}_t, \bar{h}_t) \quad (16)$$

where f is a function that merges the two output sequences of the BLSTM layers, this technique for merging can be a concatenation, summation, averaging, or multiplication. Within the Keras API, this merge method can be set through the 'merge mode' parameter in the Bidirectional layer, with the default value being concatenation. The concatenated output of the BLSTM layers is a vector containing the feature representation for each amino acid. Concatenation is different from summation and has been shown to preserve more information.

The 2 BLSTM layers and the previous convolutional features were stacked and concatenated with each other.

$$h = \text{concat}(\bar{h}_t^1, \bar{h}_t^2, c) \quad (17)$$

where $\bar{h}_t^1$ and $\bar{h}_t^2$ are the two stacked BLSTM layers, and c is the convolved feature from the convolutional layers.

Any of the other specific LSTM layer parameters were maintained at their default values throughout training.

3.5. Fully connected layers

Following the convolutional feature extraction and encoding, the recurrent LSTM layers were a series of fully connected dense layers. These dense layers, also known as Multi-Layer Perceptron (MLP), perform a non-linear transformation on the previous layer, where every input is connected to every output by a weight, thus making the layers fully connected [86].

The dense layers can also reduce the dimensionality of the feature map generated from the convolutions along with the number of parameters. This reduction in dimensionality and parameters can reduce the feature space and remove redundant features, helping in the final sequence classification. Dimensionality reduction has been shown to positively affect protein classification problems [87]. The dense layer has a weight matrix W , a bias vector B , and an activation function (ReLU). Dense layers follow a similar formula to linear layers:

$$f c^2 = f(W * x + b) \quad (18)$$

where $f c^2$ is the output from the second fully connected layer in the model, f is a linear/non-linear activation function, x is the input, and b is the bias vector. In our model, dense fully connected layers were placed after the three convolutional layers and after the LSTM recurrent layers. Another dense layer was also used as the final output layer, with eight nodes representing the eight classifications of the protein secondary structure; here, the Softmax activation function was used.

3.6. Activation functions

An activation function calculates a weighted sum of its input, adds a bias, and decides whether the neuron in the network should be activated or not, determining the output from one layer to the next [88]. Fig. 4 shows the role of an activation function and its position within a neural network.

Four main activation functions were used throughout the final network: Rectified Linear Unit (ReLU), Sigmoid, Tanh, and Softmax. These functions are defined below, along with their use and role within the network.

3.6.1. Rectified Linear Unit (ReLU)

ReLU thresholds values at 0, i.e., $f(x) = \max(0, x)$, meaning that it outputs 0 when $x < 0$ and outputs a linear function when $x \geq 0$ [89]. ReLU is favored in deep learning due to its condensed computational complexity and ability to produce activations for large sequences/inputs, generating better performance compared to sigmoid and tanh [90]. Although, a limitation of ReLU is that it can easily overfit compared to the sigmoid function. To mitigate this overfitting, the dropout technique has been implemented to lessen its effect [91]. Dropout is discussed further in the Model Optimization section below. The ReLU equation is expressed as:

$$f(x) = \max(0, x) \begin{cases} x_i, & \text{if } x_i \geq 0 \\ 0, & \text{if } x_i < 0 \end{cases} \quad (19)$$

In our model, ReLU was used as the activation function within the convolutional layers and for the dense fully connected layers.

3.6.2. Sigmoid

The sigmoid or logistical function is an activation function where its input is transformed into a continuous range of values between 0 and 1. Values greater than one are set to one, and values much smaller than 0 are transformed to 0 [88]. The sigmoid function is expressed as:

$$f(x) = \frac{1}{1 + \exp(-x)} \quad (20)$$

In the network, the sigmoid activation function was used within the recurrent component of the network, specifically the LSTM layers. Sigmoid is widely used as the gating function within LSTM nodes for the three gates. As the function's output is between 0 and 1, it determines the binary decision of whether to allow complete flow or no flow of information throughout the gates [76]. This can be seen in Fig. 3, showing the three sigmoid functions for the three gates.

3.6.3. Tanh

The tanh, referred to as the hyperbolic tangent function, is the ratio of the corresponding hyperbolic sine and hyperbolic cosine function [88]. The tanh function is expressed as:

$$f(x) = \left(\frac{e^x - e^{-x}}{e^x + e^{-x}} \right) \quad (21)$$

In the network, the tanh activation function was used as the activation function for the recurrent component of the network, specifically the bidirectional LSTM layers.

3.6.4. Softmax

Softmax normalizes the output of a network to a probability distribution of the multiple target classes. This normalized probability distribution consists of Y probabilities that are proportional to the exponentials of the input values, where Y represents the number of classes in an input vector Z . Each output value in Y will range from 0 to 1, summing to 1, allowing classes to be interpreted as probabilities. The final output is the class with the highest probability [88]. The Softmax function is expressed as:

$$\sigma(x_i) = \left(\frac{\exp(x_i)}{\sum_{j=1}^K \exp(x_j)} \right) \quad (22)$$

Table 3
Comparison of DCBLSTM results to recent popular methods, using the CB513, CASP10, and CASP11 test datasets.

Method/Model	CASP10	CASP11	CB513
RaptorX-SS8(CNF)[94]	–	–	65% (Q8)
ICML2014[50]	–	–	66.4%(Q8)
DeepCNF (2018)[13]	84.4%(Q3) 71.8%(Q8)	84.7%(Q3) 72.3%(Q8)	82.3%(Q3) 68.3%(Q8)
MUFOLD-SS (2018)[51]	86.49%(Q3) 76.47%(Q8)	85.20%(Q3) 74.51%(Q8)	– 70.63%(Q8)
CRRNN (2018)[19]	84.2%(Q3) 73.8%(Q8)	82.6%(Q3) 71.6%(Q8)	86.1%(Q3) 71.4%(Q8)
eCRRNN (2018)[19]	85.9%(Q3) 76.3%(Q8)	83.7%(Q3) 73.9%(Q8)	87.8%(Q3) 74.0%(Q8)
CNNH PSS (2018)[55]	–	–	70.3%(Q8)
DeepACLSTM (2019)[12]	75.0%(Q8)	73.0%(Q8)	70.5%(Q8)
F1DCNN-SS (2020)[95]	74.9%(Q8)	71.3%(Q8)	70.5%(Q8)
DNSS2 (2021)[96]	–	–	73.36%(Q8)
MLPRNN (2021)[97]	–	–	70.59%(Q8)
NetSurfP-3.0 (2022)[98]	–	–	84.6%(Q8)
MCNN-PSSP (2022)[99]	74.9%(Q8)	71.5%(Q8)	70.6%(Q8)
CGAN-PSSP (2022)[99]	74.6%(Q8)	71.3%(Q8)	70.3%(Q8)
CBi-GRU (2024)[100]	76.2%(Q8)	76.4%(Q8)	–
DCBLSTM	83.9% (Q8)	84.3%(Q8)	88.9%(Q8)

where $\exp(x_i)$ is the standard exponential function for input vector x_i and $\exp(x_j)$ is standard exponential function for output vector. K is the number of classes in the multi-class classifier.

Softmax is the final activation function in the output layer of many deep learning networks created for multi-class classification [92]. It also features commonly in networks similar to ours and PSP-related problems [12,55,56,66,93]. Thus, in our network, we used Softmax as the final activation function at the output layer.

4. Results

The DCBLSTM optimized network achieved an accuracy of 88.9% on the widely used CB513 test dataset, 83.9% accuracy on CASP10, and 84.3% on CASP11. The evaluation metrics captured with this model are detailed in the next section.

The results obtained from the model equal or marginally surpass some popular benchmarks and similar methods for PSP; this can be seen in Table 3.

5. Discussion

This section presents an in-depth analysis of model optimization and hyperparameters. It also highlights the novelty and uniqueness of the model through various comparative studies. Further, it contains a detailed discussion of the employed evaluation metrics. The training, testing, and validation datasets utilized are outlined, along with the methodology adopted for parameter tuning. Finally, potential avenues for future research are explored.

The final model consists of an initial CNN component and an RNN component (in the form of two bi-directional LSTM layers) with two deep, fully connected layers. The network consists of many tunable hyperparameters, as discussed in the Model Optimization section.

5.1. Comparative advantages of DCBLSTM

The proposed DCBLSTM model introduces several distinct advantages and novel contributions to the field of protein secondary structure prediction, particularly in comparison with recent state-of-the-art methodologies.

Firstly, the DCBLSTM model demonstrates superior performance on standard benchmarks. Notably, the DCBLSTM model achieves an 88.9% Q8 accuracy on the CB513 dataset. This performance significantly surpasses that of other recent state-of-the-art methods, including NetSurfP-3.0 (84.6%) [98], MCNN-PSSP (70.6%) [99], and CBi-GRU (70.4%) [100]. Furthermore, on the CASP10 and CASP11 datasets, DCBLSTM records 83.9% and 84.3% Q8 accuracy, respectively. These figures represent a substantial improvement over models such as CBi-GRU (76.2% on CASP10, 76.4% on CASP11) [100], MCNN-PSSP (74.9% on CASP10, 71.5% on CASP11) [99], and DeepCNF (71.8% on CASP10, 72.3% on CASP11) [13]. A detailed comparison is presented in Table 3.

Secondly, the DCBLSTM model features an efficient architecture that achieves state-of-the-art results without relying on large, computationally intensive pre-trained language models, such as NetSurfP-3.0. This design choice highlights a pathway to high accuracy while maintaining computational feasibility, which is crucial for broader applicability and deployment in resource-constrained environments.

Thirdly, the model incorporates a sophisticated multi-scale feature extraction mechanism. It employs three distinct convolutional layers with progressively increasing filter and kernel sizes. This hierarchical design allows the model to simultaneously capture local dependencies at varying granularities, from fine-grained sequence motifs to broader contextual patterns. This contrasts with methods that utilize fixed-scale convolutional filters or less varied multi-scale approaches, offering a more comprehensive understanding of local sequence information. While some recent models, like CBi-GRU, also incorporate multiple kernel sizes (e.g., 3, 7, 11), the specific configuration and integration within DCBLSTM notably contribute to its enhanced performance.

Fourthly, a novel and optimized regularization strategy is a key component of DCBLSTM's robustness. This involves the application of carefully tuned dropout rates at strategic points within the network: 0.2 after the convolutional layers, 0.5 within the LSTM layers, 0.4 after feature concatenation, and 0.3 before the final output layer. This layered and differential dropout strategy is specifically designed to prevent overfitting in the context of PSSP, thereby significantly enhancing the model's generalization capabilities across diverse and unseen protein sequences.

Fifthly, the DCBLSTM architecture uniquely achieves an optimal balance between local and global feature capture. Convolutional Neural

Table 4
Final values for all Hyperparameters.

Parameter	Value
Learning rate	0.0015
Batch size	120
Optimizer	Adam
Dropout	1. After each convolutional layer: 0.2 2. Within the recurrent LSTM layers: 0.5 3. After the feature concatenation: 0.4 4. Prior to the final dense classification output layer: 0.3
Kernel Regularizer	None
Kernel Initializer	Glorot Uniform initialization
Early Stopping	Initial: 150 epochs; After Implementation: 60 epochs

Networks (CNNs) are effectively utilized to extract localized sequence patterns, while Bidirectional Long Short-Term Memory (BLSTM) layers are employed to model complex, long-distance interactions within the protein sequence. This synergistic integration allows the model to process both immediate and distant contextual information comprehensively. This approach has demonstrated superior performance compared to certain transformer-based models, which may sometimes struggle with capturing fine-grained local patterns, and various graph-based methods, which can be computationally demanding when applied to large-scale protein datasets. Although other hybrid models, such as CBi-GRU, also combine CNNs and recurrent units (Bi-GRUs) for similar objectives, the specific architectural refinements and optimizations within DCBLSTM contribute to its overall superior predictive accuracy.

Finally, the development and refinement of the DCBLSTM model greatly benefited from access to the PARAM Utkarsh supercomputing infrastructure. This resource enabled an extensive and thorough hyperparameter optimization process, directly contributing to the model's robust performance and stability.

This combination of architectural innovation, optimized regularization, and robust training procedures positions DCBLSTM as a leading method in protein secondary structure prediction, demonstrating significant advancements in both accuracy and efficiency.

5.2. Model optimization and hyperparameters

A neural network's performance heavily relies on both its architecture and the optimization of its hyperparameters. These hyperparameters include a range of parameters such as batch size, learning rate, dropout, optimizer, kernel regularizer, and kernel initializer. Fine-tuning these hyperparameters through rigorous testing is crucial for achieving the highest accuracy and efficiency in the model's outcomes [101]. The optimal values for each hyperparameter, determined after extensive experimentation, are documented here, highlighting their impact on the neural network's functionality. Table 4 presents the final values for each hyperparameter used within the network. It is important to note that any parameter within a layer not explicitly mentioned retains its default setting, as specified in the Keras documentation. This comprehensive approach to hyperparameter optimization not only enhances model performance but also provides valuable insights into the intricate workings of neural networks, laying the groundwork for further advancements in the field.

5.2.1. Learning rate

A network's learning rate (LR) indicates the speed at which it learns. The optimal LR is found when the model's loss stops decreasing and starts to increase, though this method can be cumbersome. Alternatives include using cyclic LR schedules, triangular methods, and cosine annealing to adjust the LR [102,103].

Initially, when designing and training the network, a low LR of 0.003 was selected based on similar networks designed for the PSP

problem [12,104]. To maintain optimal network performance, we utilized a TensorFlow callback, ReduceLROnPlateau, which lowers the learning rate (LR) when there is no improvement in a chosen metric, typically the validation loss.

In the later stages of development, batch normalization was added to the model. Adding this layer meant that the network converged more quickly, and the gradients reduced less rapidly, thus allowing for higher learning rates [105]. The model was then evaluated with varying learning rates using the process mentioned in the Parameter Optimization section. With batch normalization applied, the final LR was reduced to 0.0015 within the Adam optimizer.

5.2.2. Batch size

Batch size defines the number of training samples propagated during one training iteration of the network. From Masters et al.'s findings, it was determined that smaller batch sizes enhance training stability, generalization performance, and reduce memory usage across various experiments [106].

Smaller batch sizes also help mitigate the 'generalization gap' associated with larger ones, which can negatively impact generalization performance by causing convergence to sharp minima of the training function [107,108]. Smith et al. concluded that marginal returns begin to diminish when the mini-batch size is increased [109]. Finally, a batch size of 120 was decided, which offered a balance of generalization and convergence.

5.2.3. Batch normalization

The aim is to limit the covariance shift between hidden unit values, enhancing network performance and stability. It normalizes the output of a preceding activation layer by subtracting the batch mean and dividing it by the batch's standard deviation [105].

Batch normalization also adds noise to reduce overfitting, improving model performance and lowering generalization error [110]. This regularization property of batch normalization works similarly to dropout, meaning that dropout within the network can be reduced. In the network, the dropout in the convolutional layers was reduced from 0.4 to 0.2 and the dense layers to 0.4 and 0.3, respectively, from their original value of 0.5 preceding batch normalization.

5.2.4. Optimizer

Optimizers are the bridge between the model's parameters, such as weights and learning rates, and the loss function. The optimizers investigated in this paper include Stochastic Gradient Descent (SGD), RMSprop, Adamax, Adadelat, and AdaGrad. The chosen optimizer for our model was Adam.

Adam is easy to implement and is well-suited for problems involving large datasets or numerous parameters. It is particularly effective for data that is very noisy or has sparse gradients. It is computationally efficient and has little memory requirements [111]. Due to these advantages, it was chosen in the final model implementation.

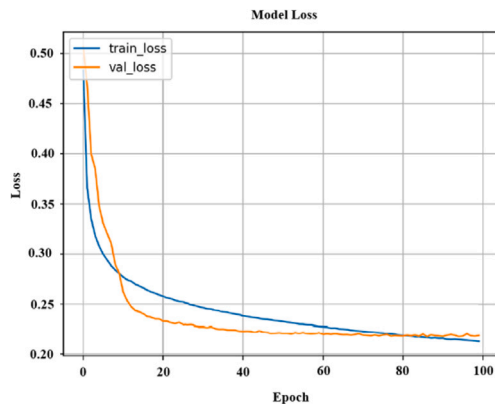


Fig. 5. Train vs validation loss for the model showcasing overfitting.

5.2.5. Dropout

Dropout is the procedure to drop units randomly during training time, meaning that all inputs and outputs to a neuron will be disabled at the current training iteration [112]. The dropout rate is the probability at which the layer outputs are dropped out [112].

Fig. 5 shows that the model starts overfitting after 80 epochs, i.e., when validation loss increases but training loss decreases, meaning that the model memorizes the training data. In our network, dropout was used in four primary areas of the network: after each convolutional layer, within the recurrent LSTM layers, after the feature concatenation, and prior to the final dense classification output layer. In the CNN layers, a low dropout of 0.2 was used. This low dropout can be attributed to including a batch normalization layer after each convolution. Dropout and batch normalization, when used together, can lead to an increase in variance shift, with dropout shifting the variance and batch normalization maintaining the variance. A low dropout is used to mitigate this variance shift [113]. Within the CNN components of the network, the dropout was placed following the ReLU activation layer. The output of the dropout was then fed into the next convolutional layer [112].

The recurrent LSTM layers have a normal dropout parameter, which is the proportion of units to drop for the linear transformation of the inputs, and a recurrent dropout parameter, which is the proportion of units to drop for the linear transformation of the recurrent state. Both the normal and recurrent dropouts were set to 0.5. The dropout after the convolutional feature concatenation was set to 0.4, and prior to the final output layer, the dropout was set to 0.3. The comparatively low dropout used throughout the model can be attributed to the effect of the other regularization and optimization techniques implemented.

5.2.6. Kernel regularizer

A kernel regularizer penalizes the layer's kernel during optimization, preventing overfitting by reducing the coefficients of the predictors. These penalties are incorporated into the loss function that the network optimizes. The most commonly used regularizers are L1 (Lasso regression) and L2 (Ridge regression) [114,115].

L1 and L2 regularization have been utilized in PSP problems, where results have been shown to improve when using L2 regularization, reducing the likelihood of overfitting [13]. L1 regularization has also shown promise in certain PSP problems, including understanding protein dynamics using Hidden Markov Models [116].

In our study, L1 and L2 regularization were evaluated across various network layers. While L2 slightly improved model accuracy and stability compared to L1, the gains were minimal. Ultimately, it was decided not to include either regularization in the final model, as other methods like dropout and early stopping were already in place, which could lead to over-regularization. Additionally, batch normalization and max pooling were used to reduce overfitting.

5.2.7. Kernel initializers

A kernel or weight initializer determines how the initial weights are set, specifying the statistical distribution or function used for this purpose [114]. We opted for random initialization, which proved to be the most effective method compared to other weight initialization strategies. The Glorot Uniform method was employed as the kernel initializer across convolutional, recurrent, and dense layers. Other initializers, such as RandomNormal, RandomUniform, TruncatedNormal, Zeros, Ones, and GlorotNormal, were also evaluated for their performance in various layers of the network.

In the recurrent LSTM layers, an orthogonal initializer is used for the recurrent kernel weights matrix to transform the recurrent state linearly. This orthogonal matrix helps mitigate vanishing and exploding gradient issues, preserving gradient norm and enabling more effective backpropagation [116].

5.2.8. Early stopping

Early stopping allows for the training to halt when overfitting begins, accuracy begins to decrease. This callback can be implemented through Keras. A monitoring variable is set (default is the validation loss), which will be monitored throughout training, and if this variable surpasses a certain threshold ('baseline'), then training will be halted [1]. During the original training of the model, the number of epochs was set to 150. However, early stopping stopped the model after 60 epochs, saving overfitting of the model and a significant amount of compute time and resources.

5.3. Evaluation metrics

The Keras API used to create the network offers a plethora of available metrics that help understand the performance of a model. These metrics were captured in two different instances: fitting the model and evaluating the model on the test datasets. The metrics captured during the training included accuracy, loss, mean squared error, mean absolute error, precision, and recall. Additionally, evaluation loss, accuracy and the F1 score were calculated from the test datasets. The accuracy evaluation metric is referred to as the Q8 accuracy due to the eight categories in PSSP, so this metric shows the percentage of amino acid residues correctly predicted. Q8 accuracy is a popular metric used in many PSP problems. [12,19,47,55,94].

5.3.1. Accuracy

Accuracy calculates how often predictions equal labels, i.e., how often the model predicts the correct secondary protein structure. In the following equations for Accuracy as well as other metrics, TP, TN, FP, FN refer to True Positives, True Negatives, False Positives, and False negatives respectively:

$$accuracy = \frac{\Sigma(TP) + \Sigma(TN)}{\Sigma(TP) + \Sigma(TN) + \Sigma(FP) + \Sigma(FN)} \quad (23)$$

5.3.2. Loss

The loss class calculates the loss of the network based on the pre-determined loss function, which was categorical cross-entropy:

$$loss = \sum_{c=1}^M y_{o,c} \log(p_{o,c}) \quad (24)$$

where M is the number of classes, c is the class label, o is the observation, y is a binary indicator (0,1) if the class label is correction classification, and p is the predicted probability observation o of class c .

5.3.3. Mean squared error

The mean squared error calculates the average of the squares of the errors between the target and predicted values.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (25)$$

where n is the number of data points, y_i is the observed value and $\hat{y}_i$ is the predicted value.

Table 5
Evaluation Metrics from the test datasets.

Metric	CB513	CASP10	CASP11
Accuracy	88.9%	83.9%	84.3%
Loss	0.326	0.580	0.511
Mean Squared Error	0.069	0.088	0.085
Mean Absolute Error	0.071	0.090	0.086
Recall	0.9772	0.9572	0.9578
Precision	0.9770	0.9563	0.9579

5.3.4. Mean absolute error

The mean absolute error measures the absolute difference between target and predicted values, where all individual differences have equal weights:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (26)$$

where n is the number of data points, y_i is the observed value and $\hat{y}_i$ is the predicted value.

5.3.5. Precision

Precision is the proportion between correctly predicted predictions in relation to all positive predictions:

$$precision = \frac{\Sigma(TP)}{\Sigma(TP) + \Sigma(FP)} \quad (27)$$

5.3.6. Recall

Recall (also known as sensitivity) is the proportion of correctly predicted (True Positives) predictions that were identified by the model from all the actual positive samples in the dataset:

$$recall = \frac{\Sigma(TP)}{\Sigma(TP) + \Sigma(FN)} \quad (28)$$

The models were evaluated, and the above metrics were captured using the primary test datasets CB513, CASP10, and CASP11. The resultant metrics from the DCBLSTM models on each of these test datasets can be seen in Table 5.

5.4. Parameter tuning implementation

As discussed in the earlier sections, optimizing the hyperparameters of our final network architecture was crucial to achieving optimal performance. Given the computational complexity of PSP networks, we employed a parallel grid search approach to efficiently explore a wide range of hyperparameter configurations. This involved conducting multiple concurrent trials, each testing a unique combination of discrete, categorical, and range-based hyperparameters. In each trial, a random selection from these values was made to configure and train a model instance. The performance of each trial was evaluated using a specific metric, with the aim of maximizing accuracy on the test dataset. The highest-performing trial's hyperparameter configuration was selected as the final setup for our model. To expedite this process, we leveraged the high-performance computing capabilities of Param Utkarsh.

5.5. Training, testing, and validation data

In our models, we utilized the CullPDB 6133 dataset from PISCES CullPDB, which includes 6128 protein sequences, their amino acids, and 57 features per residue. A filtered version of this dataset, with redundancy removed to avoid overlap with the CB513 dataset, was employed. Consequently, our final training set comprised 5534 protein sequences, divided into 5278 for training and 256 for validation.

Three publicly available test datasets were used in evaluating our models: CASP10 [6], CASP11 [7], and CB513 [8]. These datasets have been previously used in other PSP methods [12,13,19,51,55,56]. The main test set used was CB513, which consists of 514 protein sequences and 87,041 total residues in the sequences. The CASP10 and CASP11 datasets consist of 22,041 and 20,498 residues, respectively.

5.6. Pharmacogenomic approaches Very standard use cas

Pharmacogenomics aims to tailor drug treatments to individuals based on their genetic makeup, which directly impacts the structure and function of proteins involved in drug metabolism, efficacy, and toxicity. One of the primary applications of protein secondary structure prediction in pharmacogenomics is understanding how genetic variations (e.g., single nucleotide polymorphisms, SNPs) affect the structure and function of key proteins, such as drug-metabolizing enzymes (e.g., cytochrome P450 family), drug transporters, and receptors. The DCBLSTM model can play a critical role in several ways:

1. **Prediction of Structural Changes due to Mutations** In pharmacogenomics, genetic variations can lead to amino acid substitutions, which may alter the secondary and tertiary structures of proteins. Our DCBLSTM model, which excels at capturing both local and long-range dependencies in protein sequences, can be used to predict how these mutations affect the formation of alpha-helices, beta-sheets, or loops. This information can be crucial in understanding how structural changes impact protein function and, consequently, drug interactions.

2. **Drug-Target Interaction** Many drugs exert their effects by binding to specific protein targets. The secondary structure of a protein determines its folding and active sites, which are essential for drug binding. The DCBLSTM model can assist in predicting the secondary structures of drug targets, especially when experimental structures are unavailable. This can provide insights into how variations in protein folding might influence the binding affinity of drugs, helping to explain individual variability in drug response.

3. **Prediction of Protein-Drug Metabolism Enzymes** Proteins such as CYP450 enzymes are responsible for the metabolism of many drugs. By predicting the secondary structure of these enzymes under different genetic variants, the DCBLSTM model can help pharmacogenomics researchers understand how mutations might alter enzyme activity or stability, influencing drug metabolism rates and potentially leading to adverse drug reactions.

4. **Optimizing Drug Design** In drug discovery, predicting the structure of a protein target is essential for rational drug design, particularly in structure-based drug design (SBDD). The DCBLSTM model could be used to predict secondary structures of drug targets in cases where high-resolution crystal structures are unavailable, allowing researchers to generate models for virtual screening or de novo drug design. By understanding the secondary structure, we can also predict binding pockets and allosteric sites, which are crucial for designing effective and selective drugs.

5.7. Biological significance and insights

The DCBLSTM model, combining convolutional layers with bidirectional long short-term memory (BLSTM) units, excels at capturing both local and long-range dependencies in amino acid sequences. Through our analysis, several key patterns contributing to the model's performance were identified.

The convolutional layers are highly effective in detecting short-range sequential patterns in the primary amino acid sequence. These patterns often correlate with the formation of alpha-helices and beta-sheets, which are stabilized by hydrogen bonds between the backbone atoms of amino acids that are close in sequence. For example, the model frequently identifies specific motifs like glycine-rich or proline-rich regions, which are known to disrupt regular secondary structures and introduce bends or loops.

The BLSTM component of the model is crucial for capturing long-range interactions in the sequence that simpler models often overlook. These long-range dependencies are especially significant for beta-sheets, where hydrogen bonds form between amino acids that are distant in sequence but close in the three-dimensional structure. By considering both forward and backward sequence contexts, the BLSTM enhances its ability to predict beta-strands, which often require information from non-adjacent regions of the sequence.

The DCBLSTM model revealed several biologically relevant insights during the study. For helix-forming residues, the model demonstrated higher accuracy in predicting helices for amino acids such as alanine, leucine, and glutamine. These residues are commonly involved in helix formation, indicating that the model has effectively learned to recognize amino acid propensities for forming specific secondary structures, which aligns with established biological patterns.

Similarly, beta-sheet prediction was notably enhanced for residues like valine, isoleucine, and phenylalanine, which have a higher tendency to form beta-strands. The model was able to correctly identify beta-strands even in cases where the primary sequence alone did not strongly indicate the structure. This finding suggests that the BLSTM layers effectively captured critical long-range interactions necessary for beta-sheet formation.

The model also performed well in predicting coil regions, particularly in sequences containing proline-rich motifs. Proline's rigid ring structure disrupts alpha-helices and beta-sheets, leading to loops and turns. The model consistently identified such regions accurately, demonstrating its ability to handle irregular patterns in protein sequences.

By capturing both local and global sequence features, the DCBLSTM model provides biologically interpretable predictions that align with known secondary structure-forming patterns. These insights enhance our understanding of how specific primary sequence motifs contribute to secondary structure formation. These findings have implications for protein folding studies and could inform future research on protein design and engineering.

5.8. Future work

The DCBLSTM model demonstrated promising results for PSSP, as detailed in the Results section. These performance metrics either meet or exceed those of contemporary and established methods like TE_SS (79.08% Q8 accuracy) and CRRNN (73% Q8 accuracy), highlighting the model's effectiveness. The success of the DCBLSTM architecture can be attributed to its ability to effectively capture both local and long-distance dependencies within protein sequences. The initial convolutional layers helped extract local features and relationships between adjacent amino acids. The subsequent bidirectional LSTMs modeled long-range dependencies, capturing interactions between distant residues that influence protein folding.

Despite achieving competitive accuracy, the model showed a slight advantage on the CB513 dataset compared to CASP10 and CASP11, suggesting possible room for refinement, particularly in addressing training data biases or variations in protein structure distributions. Future work could benefit from exploring dataset balancing techniques and evaluating the model on newer CASP editions (CASP 13 and 14) to test its adaptability to unseen structures. One should also consider the balance between model accuracy and computational efficiency. The current focus was not on reducing computational demand, but future research could explore more streamlined architectures or dimensionality reduction strategies, especially where faster prediction times are paramount.

6. Conclusion

PSP remains a pivotal area of study in bioinformatics, essential for understanding protein functionality in drug design. Despite consider-

able advances over the last six decades, challenges persist, especially in capturing long-range dependencies within protein sequences. Recent advancements have benefited from the continued incorporation of Machine learning and Deep learning.

From extensive experimentation, the most effective model, DCBLSTM, utilized a three-layer 1-dimensional CNN to identify local amino acid interactions, followed by two bidirectional LSTMs for mapping long-range dependencies. This configuration emerged after thorough hyperparameter optimization, leveraging techniques like batch normalization, dropout, kernel initialization and the Adam optimization algorithm for enhanced performance. The DCBLSTM model achieved impressive 8-state accuracy rates of 88.9%, 83.9%, and 84.3% on the CB513, CASP10, and CASP11 datasets, respectively, underscoring the significance of blending local and long-distance information for accurate PSP. These results point to the crucial role of Deep Learning in PSP research.

The DCBLSTM model demonstrated strong performance for Protein Secondary Structure Prediction (PSSP), with competitive accuracy rates compared to established methods like TE_SS and CRRNN. However, its slightly better performance on the CB513 dataset than CASP10 and CASP11 suggests potential limitations in generalizability due to training data biases or protein structure distribution variations. Future work could explore balancing datasets and refining the architecture to enhance both adaptability and computational efficiency, particularly for applications requiring faster prediction times.

7. Abbreviations

PSP—Protein Structure Prediction
 RNN—Recurrent Neural Network
 ANN—Artificial Neural Network
 CNN—Convolutional Neural Network
 DNN—Deep Neural Network
 BLSTM—Bidirectional Long Short-Term Memory
 BGRU—Bidirectional Gated Recurrent Unit
 ReLU—Rectified Linear Unit
 MLP—Multi-layer Perceptron
 NLP—Natural Language Processing
 MSE—Mean Squared Error
 MAE—Mean Absolute Error
 LR—Learning Rate
 PARAM—PARAllel Machine

CRedit authorship contribution statement

Suvidhi Banthia: Writing – original draft, Methodology, Formal analysis. **Adam McKenna:** Writing – original draft, Methodology, Conceptualization. **Shailendra Kumar Tiwari:** Writing – review & editing. **Sandhya P.N. Dubey:** Writing – review & editing, Writing – original draft, Supervision, Investigation, Conceptualization.

Ethics approval and consent to participate

Not applicable.

Funding sources

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We acknowledge National Supercomputing Mission (NSM) for providing computing resources of 'PARAM UTKARSH' at CDAC Knowledge Park Bangalore, which is implemented by C-DAC and supported by the Ministry of Electronics and Information Technology (MeitY) and Department of Science and Technology (DST), Government of India. We also thank Dr. Shankar Lal Jatav, Project Engineer at CDAC, Pune, for his assistance in operating PARAM UKARSH supercomputer.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.combiomed.2025.110457>.

Data availability

The dataset used to support the findings of this research can be found here <https://github.com/SuvidhiBanthia/DCBLSTM>.

References

- [1] P. Radivojac, W.T. Clark, T.R. Oron, A.M. Schnoes, T. Wittkop, A. Sokolov, K. Graim, C. Funk, K. Verspoor, A. Ben-Hur, et al., A large-scale evaluation of computational protein function prediction, *Nature Methods* 10 (3) (2013) 221–227, <http://dx.doi.org/10.1038/nmeth.2340>.
- [2] Y. Zhou, M. Karplus, Interpreting the folding kinetics of helical proteins, *Nature* 401 (6751) (1999) 400–403, <http://dx.doi.org/10.1038/43937>.
- [3] S.B. Ozkan, G.A. Wu, J.D. Chodera, K.A. Dill, Protein folding by zipping and assembly, *Proc. Natl. Acad. Sci.* 104 (29) (2007) 11987–11992, <http://dx.doi.org/10.1073/pnas.0703700104>.
- [4] H. Zhou, Y. Zhou, SPEM: improving multiple sequence alignment with sequence profiles and predicted secondary structures, *Bioinformatics* 21 (18) (2005) 3615–3621, <http://dx.doi.org/10.1093/bioinformatics/bti582>.
- [5] X. Deng, J. Cheng, MSACompro: protein multiple sequence alignment using predicted secondary structure, solvent accessibility, and residue-residue contacts, *BMC Bioinformatics* 12 (2011) 1–16, <http://dx.doi.org/10.1186/1471-2105-12-472>.
- [6] B. Li, V.G. Krishnan, M.E. Mort, F. Xin, K.K. Kamati, D.N. Cooper, S.D. Mooney, P. Radivojac, Automated inference of molecular mechanisms of disease from amino acid substitutions, *Bioinformatics* 25 (21) (2009) 2744–2750, <http://dx.doi.org/10.1093/bioinformatics/btp528>.
- [7] P. Yue, Z. Li, J. Moul, Loss of protein structure stability as a major causative factor in monogenic disease, *J. Mol. Biol.* 353 (2) (2005) 459–473, <http://dx.doi.org/10.1016/j.jmb.2005.08.020>.
- [8] S. Khan, M. Vihinen, Spectrum of disease-causing mutations in protein secondary structures, *BMC Struct. Biol.* 7 (2007) 1–18, <http://dx.doi.org/10.1186/1472-6807-7-56>.
- [9] R. Fitzsimmons, N. Amin, V.N. Uversky, Understanding the roles of intrinsic disorder in subunits of hemoglobin and the disease process of sickle cell anemia, *Intrinsically Disord. Proteins* 4 (1) (2016) e1248273, <http://dx.doi.org/10.1080/21690707.2016.1248273>.
- [10] W. Lee, P. Yue, Z. Zhang, Analytical methods for inferring functional effects of single base pair substitutions in human cancers, *Hum. Genet.* 126 (2009) 481–498, <http://dx.doi.org/10.1007/s00439-009-0677-y>.
- [11] W. Kabsch, C. Sander, Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features, *Biopolym.: Orig. Res. Biomol.* 22 (12) (1983) 2577–2637, <http://dx.doi.org/10.1002/bip.360221211>.
- [12] Y. Guo, W. Li, B. Wang, H. Liu, D. Zhou, DeepACLSTM: deep asymmetric convolutional long short-term memory neural models for protein secondary structure prediction, *BMC Bioinformatics* 20 (2019) 1–12, <http://dx.doi.org/10.1186/s12859-019-2940-0>.
- [13] S. Wang, J. Peng, J. Ma, J. Xu, Protein secondary structure prediction using deep convolutional neural fields, *Sci. Rep.* 6 (1) (2016) 1–11, <http://dx.doi.org/10.1038/srep18962>.
- [14] X. Zhang, J. Zhao, Y. LeCun, Character-level convolutional networks for text classification, *Adv. Neural Inf. Process. Syst.* 28 (2015) <http://dx.doi.org/10.48550/arXiv.1509.01626>.
- [15] T.N. Sainath, O. Vinyals, A. Senior, H. Sak, Convolutional, long short-term memory, fully connected deep neural networks, in: 2015 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, Ieee, 2015, pp. 4580–4584, <http://dx.doi.org/10.1109/ICASSP.2015.7178838>.
- [16] D.T. Jones, T. Singh, T. Kosciolk, S. Tetchner, MetaPSICOV: combining co-evolution methods for accurate prediction of contacts and long range hydrogen bonding in proteins, *Bioinformatics* 31 (7) (2015) 999–1006, <http://dx.doi.org/10.1093/bioinformatics/btu791>.
- [17] A. Graves, A.-r. Mohamed, G. Hinton, Speech recognition with deep recurrent neural networks, in: 2013 IEEE International Conference on Acoustics, Speech and Signal Processing, Ieee, 2013, pp. 6645–6649, <http://dx.doi.org/10.1109/ICASSP.2013.6638947>.
- [18] Y. Pan, T. Mei, T. Yao, H. Li, Y. Rui, Jointly modeling embedding and translation to bridge video and language, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 4594–4602, <http://dx.doi.org/10.48550/arXiv.1505.01861>.
- [19] B. Zhang, J. Li, Q. Lü, Prediction of 8-state protein secondary structures by a novel deep learning architecture, *BMC Bioinformatics* 19 (2018) 1–13, <http://dx.doi.org/10.1186/s12859-018-2280-5>.
- [20] K.D. Yamada, K. Kinoshita, De novo profile generation based on sequence context specificity with the long short-term memory network, *BMC Bioinformatics* 19 (2018) 1–11, <http://dx.doi.org/10.1186/s12859-018-2284-1>.
- [21] R. Heffernan, Y. Yang, K. Paliwal, Y. Zhou, Capturing non-local interactions by long short-term memory bidirectional recurrent neural networks for improving prediction of protein secondary structure, backbone angles, contact numbers and solvent accessibility, *Bioinformatics* 33 (18) (2017) 2842–2849, <http://dx.doi.org/10.1093/bioinformatics/btx218>.
- [22] B. Alberts, A. Johnson, J. Lewis, et al., *Molecular Biology of the Cell*, fourth ed., Garland Science, New York, 2002, URL <https://www.ncbi.nlm.nih.gov/books/NBK26911>.
- [23] K.U. Linderstrøm-Lang, *Lane Medical Lectures: Proteins and Enzymes*, vol. 6, Stanford University Press, 1952.
- [24] E. Moutevelis, D.N. Woolfson, A periodic table of coiled-coil protein structures, *J. Mol. Biol.* 385 (3) (2009) 726–732, <http://dx.doi.org/10.1016/j.jmb.2008.11.028>.
- [25] R.D. Cohen, G.J. Pielak, Electrostatic contributions to protein quinary structure, *J. Am. Chem. Soc.* 138 (40) (2016) 13139–13142, <http://dx.doi.org/10.1021/jacs.6b07323>.
- [26] F. Sanger, Fractionation of oxidized insulin, *Biochem. J.* 44 (1) (1949) 126, <http://dx.doi.org/10.1042/bj0440126>.
- [27] D. Eisenberg, The discovery of the α -helix and β -sheet, the principal structural features of proteins, *Proc. Natl. Acad. Sci.* 100 (20) (2003) 11207–11210.
- [28] L. Pauling, R.B. Corey, The pleated sheet, a new layer configuration of polypeptide chains, *Proc. Natl. Acad. Sci.* 37 (5) (1951) 251–256.
- [29] J.D. Watson, F.H. Crick, Molecular structure of nucleic acids: a structure for deoxyribose nucleic acid, *Nature* 171 (4356) (1953) 737–738.
- [30] T. Simion, DNA replication, *Curr. Trends Biomed. Eng. Biosci.* 16 (2018) 555942.
- [31] US DOE Joint Genome Institute: Hawkins Trevor 4 Branscomb Elbert 4 Predki Paul 4 Richardson Paul 4 Wenning Sarah 4 Slezak Tom 4 Doggett Norman 4 Cheng Jan-Fang 4 Olsen Anne 4 Lucas Susan 4 Elkin Christopher 4 Uberbacher Edward 4 Frazier Marvin 4, RIKEN Genomic Sciences Center: Sakaki Yoshiyuki 9 Fujiyama Asao 9 Hattori Masahira 9 Yada Tetsushi 9 Toyoda Atsushi 9 Itoh Takehiko 9 Kawagoe Chiharu 9 Watanabe Hidemi 9 Totoki Yasushi 9 Taylor Todd 9, Genoscope, CNRS UMR-8030: Weissenbach Jean 10 Heilig Roland 10 Saurin William 10 Artiguenave Francois 10 Brottier Philippe 10 Bruls Thomas 10 Pelletier Eric 10 Robert Catherine 10 Wincker Patrick 10, Department of Genome Analysis, Institute of Molecular Biotechnology: Rosenthal André 12 Platzter Matthias 12 Nyakatura Gerald 12 Taudien Stefan 12 Rump Andreas 12, GTC Sequencing Center: Smith Douglas R. 11 Doucette-Stamm Lynn 11 Rubenfield Marc 11 Weinstock Keith 11 Lee Hong Mei 11 Dubois JoAnn 11, Beijing Genomics Institute/Human Genome Center: Yang Huanming 13 Yu Jun 13 Wang Jian 13 Huang Guyang 14 Gu Jun 15, et al., Initial sequencing and analysis of the human genome, *Nature* 409 (6822) (2001) 860–921, <http://dx.doi.org/10.1038/35057062>.
- [32] S. Key, J.K. Ma, P.M. Drake, Genetically modified plants and human health, *J. R. Soc. Med.* 101 (6) (2008) 290–298.
- [33] R. Saad, Discovery, development, and current applications of DNA identity testing, in: *Baylor University Medical Center Proceedings*, vol. 18, Taylor & Francis, 2005, pp. 130–133.
- [34] O. Dor, Y. Zhou, Achieving 80% ten-fold cross-validated accuracy for secondary structure prediction by large-scale training, *Proteins: Struct. Funct. Bioinform.* 66 (4) (2007) 838–845.
- [35] B. Rost, C. Sander, R. Schneider, Redefining the goals of protein secondary structure prediction, *J. Mol. Biol.* 235 (1) (1994) 13–26.
- [36] B. Rost, Protein secondary structure prediction continues to rise, *J. Struct. Biol.* 134 (2–3) (2001) 204–218.
- [37] Y. Zhou, Y. Duan, Y. Yang, E. Faraggi, H. Lei, Trends in template/fragment-free protein structure prediction, *Theor. Chem. Acc.* 128 (2011) 3–16.
- [38] K. Clark, I. Karsch-Mizrachi, D.J. Lipman, J. Ostell, E.W. Sayers, GenBank, *Nucleic Acids Res.* 44 (D1) (2016) D67–D72.
- [39] H.M. Berman, J. Westbrook, Z. Feng, G. Gilliland, T.N. Bhat, H. Weissig, I.N. Shindyalov, P.E. Bourne, The protein data bank, *Nucleic Acids Res.* 28 (1) (2000) 235–242.

- [40] K.R. Acharya, M.D. Lloyd, Protein crystallography, *Trends Pharmacol. Sci.* 1 (26) (2005) 10–14.
- [41] D.M. Gendoo, P.M. Harrison, Discordant and chameleon sequences: their distribution and implications for amyloidogenicity, *Prot. Sci.* 20 (3) (2011) 567–579.
- [42] W. Li, L.N. Kinch, P.A. Karplus, N.V. Grishin, Chseq: A database of chameleon sequences, *Prot. Sci.* 24 (7) (2015) 1075–1086.
- [43] G.E. Crooks, S.E. Brenner, Protein secondary structure: entropy, correlations and prediction, *Bioinformatics* 20 (10) (2004) 1603–1611.
- [44] D. Kihara, The effect of long-range interactions on the secondary structure formation of proteins, *Prot. Sci.* 14 (8) (2005) 1955–1963.
- [45] K. Asai, S. Hayamizu, K. Handa, Prediction of protein secondary structure by the hidden Markov model, *Bioinformatics* 9 (2) (1993) 141–146.
- [46] T. Liu, Z. Wang, MASS: predict the global qualities of individual protein models using random forests and novel statistical potentials, *BMC Bioinformatics* 21 (Suppl 4) (2020) 246.
- [47] T.-M. Yi, E.S. Lander, Protein secondary structure prediction using nearest-neighbor methods, *J. Mol. Biol.* 232 (4) (1993) 1117–1129.
- [48] G. Pollastri, D. Przybylski, B. Rost, P. Baldi, Improving the prediction of protein secondary structure in three and eight classes using recurrent neural networks and profiles, *Proteins: Struct. Funct. Bioinform.* 47 (2) (2002) 228–235.
- [49] R. Heffernan, K. Paliwal, J. Lyons, A. Dehzangi, A. Sharma, J. Wang, A. Sattar, Y. Yang, Y. Zhou, Improving prediction of secondary structure, local backbone angles and solvent accessible surface area of proteins by iterative deep learning, *Sci. Rep.* 5 (1) (2015) 11476.
- [50] J. Zhou, O. Troyanskaya, Deep supervised and convolutional generative stochastic network for protein secondary structure prediction, in: *International Conference on Machine Learning, PMLR*, 2014, pp. 745–753.
- [51] C. Fang, Y. Shang, D. Xu, MUFOLD-SS: New deep inception-inside-inception networks for protein secondary structure prediction, *Proteins: Struct. Funct. Bioinform.* 86 (5) (2018) 592–598.
- [52] A.K. Mandle, P. Jain, S.K. Shrivastava, Protein structure prediction using support vector machine, *Int. J. Soft Comput.* 3 (1) (2012) 67.
- [53] X. Li, X.-M. Pan, New method for accurate prediction of solvent accessibility from protein sequence, *Proteins: Struct. Funct. Bioinform.* 42 (1) (2001) 1–5.
- [54] A. Yaseen, Y. Li, Context-based features enhance protein secondary structure prediction accuracy, *J. Chem. Inf. Model.* 54 (3) (2014) 992–1002.
- [55] J. Zhou, H. Wang, Z. Zhao, R. Xu, Q. Lu, Cnnh_pss: protein 8-class secondary structure prediction by convolutional neural network with highway, *BMC Bioinformatics* 19 (2018) 99–109.
- [56] Z. Li, Y. Yu, Protein secondary structure prediction using cascaded convolutional and recurrent neural networks, 2016, arXiv preprint arXiv:1604.07176.
- [57] S.C. Shivaprasad, P.P. Maruthi, T.S. Venkatesh, V.K. Rajuk, Ensemble model for accuracy prediction of protein secondary structure, *Indones. J. Electr. Eng. Comput. Sci.* 32 (3) (2023) 1664–1677.
- [58] A. Kazm, A. Ali, H. Hashim, Transformer encoder with protein language model for protein secondary structure prediction, *Eng. Technol. Appl. Sci. Res.* 14 (2) (2024) 13124–13132.
- [59] T.N. Sainath, B. Kingsbury, G. Saon, H. Soltau, A.-r. Mohamed, G. Dahl, B. Ramabhadran, Deep convolutional neural networks for large-scale speech tasks, *Neural Netw.* 64 (2015) 39–48.
- [60] L. Deng, G. Hinton, B. Kingsbury, New types of deep neural network learning for speech recognition and related applications: An overview, in: *2013 IEEE International Conference on Acoustics, Speech and Signal Processing, IEEE*, 2013, pp. 8599–8603.
- [61] F. Wang, D.M. Tax, Survey on the attention based RNN model and its applications in computer vision, 2016, arXiv preprint arXiv:1601.06823.
- [62] A. Waibel, T. Hanazawa, G. Hinton, K. Shikano, K.J. Lang, Phoneme recognition using time-delay neural networks, in: *Backpropagation*, Psychology Press, 2013, pp. 35–61.
- [63] N. Kalchbrenner, E. Grefenstette, P. Blunsom, A convolutional neural network for modelling sentences, 2014, arXiv preprint arXiv:1404.2188.
- [64] Centre for Development of Advanced Computing, PARAM utkarsh supercomputer, 2024, URL <https://paramutkarsh.cdac.in/param-utkarsh-supercomputer/>.
- [65] Y. Zhao, S. Chu, Y. Zhou, K. Tu, Sequence prediction using neural network classifiers, in: *International Conference on Grammatical Inference, PMLR*, 2017, pp. 164–169.
- [66] Y. Guo, B. Wang, W. Li, B. Yang, Protein secondary structure prediction improved by recurrent neural networks integrated with two-dimensional convolutional neural networks, *J. Bioinform. Comput. Biol.* 16 (05) (2018) 1850021.
- [67] S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, D.J. Inman, 1D convolutional neural networks and applications: A survey, *Mech. Syst. Signal Process.* 151 (2021) 107398.
- [68] J. Hertz, A. Krogh, R.G. Palmer, H. Horner, Introduction to the Theory of Neural Computation, American Institute of Physics, 1991.
- [69] S. Han, Z. Meng, Z. Li, J. O'Reilly, J. Cai, X. Wang, Y. Tong, Optimizing filter size in convolutional neural networks for facial action unit recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 5070–5078.
- [70] M. Dwarampudi, N. Reddy, Effects of padding on LSTMs and CNNs, 2019, arXiv preprint arXiv:1903.07288.
- [71] L. Zaniolo, O. Marques, On the use of variable stride in convolutional neural networks, *Multimedia Tools Appl.* 79 (19) (2020) 13581–13598.
- [72] Keras Team, Conv1D Layer - Keras API, Keras.io, 2024, URL https://keras.io/api/layers/convolution_layers/convolution1d/.
- [73] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016.
- [74] O.I. Abiodun, A. Jantan, A.E. Omolara, K.V. Dada, N.A. Mohamed, H. Arshad, State-of-the-art in artificial neural network applications: A survey, *Heliyon* 4 (11) (2018).
- [75] Incf8, Structural diagrams of unidirectional and bidirectional recurrent neural networks, 2015, https://commons.wikimedia.org/wiki/File:Structural_diagrams_of_unidirectional_and_bidirectional_recurrent_neural_networks.png.
- [76] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Comput.* 9 (8) (1997) 1735–1780.
- [77] F.A. Gers, J. Schmidhuber, F. Cummins, Learning to forget: Continual prediction with LSTM, *Neural Comput.* 12 (10) (2000) 2451–2471.
- [78] G. Chevalier, LSTM Cell, 2018, URL https://en.wikipedia.org/wiki/File:LSTM_Cell.svg.
- [79] S.K. Sønderby, C.K. Sønderby, H. Nielsen, O. Winther, Convolutional LSTM networks for subcellular localization of proteins, in: *Algorithms for Computational Biology: Second International Conference, AlCoB 2015, Mexico City, Mexico, August 4–5, 2015, Proceedings 2*, Springer, 2015, pp. 68–80.
- [80] J. Schmidhuber, D. Wierstra, F.J. Gomez, Evolino: Hybrid neuroevolution/optimal linear search for sequence prediction, in: *Proceedings of the 19th International Joint Conference on Artificial Intelligence, IJCAI*, 2005.
- [81] M. Baccouche, F. Mamalet, C. Wolf, C. Garcia, A. Baskurt, Sequential deep learning for human action recognition, in: *Human Behavior Understanding: Second International Workshop, HBU 2011, Amsterdam, the Netherlands, November 16, 2011. Proceedings 2*, Springer, 2011, pp. 29–39.
- [82] M. Schuster, K.K. Paliwal, Bidirectional recurrent neural networks, *IEEE Trans. Signal Process.* 45 (11) (1997) 2673–2681.
- [83] A. Graves, J. Schmidhuber, Framewise phoneme classification with bidirectional LSTM and other neural network architectures, *Neural Netw.* 18 (5–6) (2005) 602–610.
- [84] Z. Cui, R. Ke, Z. Pu, Y. Wang, Stacked bidirectional and unidirectional LSTM recurrent neural network for forecasting network-wide traffic state with missing values, *Transp. Res. Part C: Emerg. Technol.* 118 (2020) 102674.
- [85] Wikipedia, Artificial neuron model (english), 2005, URL https://commons.wikimedia.org/wiki/File:ArtificialNeuronModel_english.png.
- [86] D. Ruck, S. Rogers, M. Kabrisky, M. Oxley, B. Suter, The multilayer perceptron as an approximation to a Bayes optimal discriminant function, *IEEE Trans. Neural Netw.* 1 (4) (1990) 296–298.
- [87] S. Mahapatra, A. Kumar, A. Sharma, S.S. Sahu, Effect of dimensionality reduction on classification accuracy for protein–protein interaction prediction, in: *Advanced Computing and Intelligent Engineering: Proceedings of ICACIE 2018, vol. 1*, Springer, 2020, pp. 3–12.
- [88] C. Nwankpa, W. Ijomah, A. Gachagan, S. Marshall, Activation functions: Comparison of trends in practice and research for deep learning, 2018, arXiv preprint arXiv:1811.03378.
- [89] V. Nair, G.E. Hinton, Rectified linear units improve restricted boltzmann machines, in: *Proceedings of the 27th International Conference on Machine Learning, ICML-10, 2010*, pp. 807–814.
- [90] M.D. Zeiler, M. Ranzato, R. Monga, M. Mao, K. Yang, Q.V. Le, P. Nguyen, A. Senior, V. Vanhoucke, J. Dean, et al., On rectified linear units for speech processing, in: *2013 IEEE International Conference on Acoustics, Speech and Signal Processing, IEEE*, 2013, pp. 3517–3521.
- [91] G.E. Dahl, T.N. Sainath, G.E. Hinton, Improving deep neural networks for LVCSR using rectified linear units and dropout, in: *2013 IEEE International Conference on Acoustics, Speech and Signal Processing, IEEE*, 2013, pp. 8609–8613.
- [92] R. Ranjan, C.D. Castillo, R. Chellappa, L2-constrained softmax loss for discriminative face verification, 2017, arXiv preprint arXiv:1703.09507.
- [93] G. Tejeshwar, S.G. Mat, A multi-layer LSTM-time-density-softmax (LDS) approach for protein structure prediction using deep learning, *Recent. Adv. Comput. Sci. Commun. (Former.: Recent. Patents Comput. Sci.)* 14 (1) (2021) 216–224.
- [94] Z. Wang, F. Zhao, J. Peng, J. Xu, Protein 8-class secondary structure prediction using conditional neural fields, in: *2010 IEEE International Conference on Bioinformatics and Biomedicine, BIBM, IEEE*, 2010, pp. 109–114.
- [95] L. Guo, Q. Jiang, X. Jin, L. Liu, W. Zhou, S. Yao, M. Wu, Y. Wang, A deep convolutional neural network to improve the prediction of protein secondary structure, *Curr. Bioinform.* 15 (7) (2020) 767–777.
- [96] Z. Guo, J. Hou, J. Cheng, DNSS2: Improved ab initio protein secondary structure prediction using advanced deep learning architectures, *Proteins: Struct. Funct. Bioinform.* 89 (2) (2021) 207–217.
- [97] Z. Lyu, Z. Wang, F. Luo, J. Shuai, Y. Huang, Protein secondary structure prediction with a reductive deep learning method, *Front. Bioeng. Biotechnol.* 9 (2021) 687426.

- [98] M.H. Høie, E.N. Kiehl, B. Petersen, M. Nielsen, O. Winther, H. Nielsen, J. Hallgren, P. Marcatili, NetSurfP-3.0: accurate and fast prediction of protein structural features by protein language models and deep learning, *Nucleic Acids Res.* 50 (W1) (2022) W510–W515.
- [99] X. Jin, L. Guo, Q. Jiang, N. Wu, S. Yao, Prediction of protein secondary structure based on an improved channel attention and multiscale convolution module, *Front. Bioeng. Biotechnol.* 10 (2022) 901018.
- [100] L. Yumeng, Protein secondary structure prediction using convolutional bidirectional GRU, *J. Math. Res.* 16 (4) (2024) 1–11.
- [101] P. Probst, A.-L. Boulesteix, B. Bischl, Tunability: Importance of hyperparameters of machine learning algorithms, *J. Mach. Learn. Res.* 20 (53) (2019) 1–32.
- [102] L.N. Smith, Cyclical learning rates for training neural networks, in: 2017 IEEE Winter Conference on Applications of Computer Vision, WACV, IEEE, 2017, pp. 464–472.
- [103] I. Loshchilov, F. Hutter, Sgdr: Stochastic gradient descent with warm restarts, 2016, arXiv preprint [arXiv:1608.03983](https://arxiv.org/abs/1608.03983).
- [104] S.K. Sønderby, O. Winther, Protein secondary structure prediction with long short term memory networks, 2014, arXiv preprint [arXiv:1412.7828](https://arxiv.org/abs/1412.7828).
- [105] S. Ioffe, C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, in: *International Conference on Machine Learning*, pmlr, 2015, pp. 448–456.
- [106] D. Masters, C. Luschi, Revisiting small batch training for deep neural networks, 2018, arXiv preprint [arXiv:1804.07612](https://arxiv.org/abs/1804.07612).
- [107] E. Hoffer, I. Hubara, D. Soudry, Train longer, generalize better: closing the generalization gap in large batch training of neural networks, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [108] N.S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, P.T.P. Tang, On large-batch training for deep learning: Generalization gap and sharp minima, 2016, arXiv preprint [arXiv:1609.04836](https://arxiv.org/abs/1609.04836).
- [109] S.L. Smith, P.-J. Kindermans, C. Ying, Q.V. Le, Don't decay the learning rate, increase the batch size, 2017, arXiv preprint [arXiv:1711.00489](https://arxiv.org/abs/1711.00489).
- [110] Y. Luo, F. Yang, Deep learning with noise, 2014, <https://www.andrew.cmu.edu/user/fanyang1/deep-learning-with-noise.pdf>.
- [111] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint [arXiv:1412.6980](https://arxiv.org/abs/1412.6980).
- [112] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, *J. Mach. Learn. Res.* 15 (1) (2014) 1929–1958.
- [113] X. Li, S. Chen, X. Hu, J. Yang, Understanding the disharmony between dropout and batch normalization by variance shift, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 2682–2690.
- [114] S. Jang, Y. Son, Empirical evaluation of activation functions and kernel initializers on deep reinforcement learning, in: *2019 International Conference on Information and Communication Technology Convergence, ICTC, IEEE*, 2019, pp. 1140–1142.
- [115] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, JMLR Workshop and Conference Proceedings*, 2010, pp. 249–256.
- [116] E. Vorontsov, C. Trabelsi, S. Kadoury, C. Pal, On orthogonality and learning recurrent networks with long term dependencies, in: *International Conference on Machine Learning*, PMLR, 2017, pp. 3570–3578.